

第3章 Linux 进程调度

- Linux 进程调度策略
- Linux 进程调度依据
- Linux 进程调度时机
- Linux 进程调度实现
- Linux 2.4 进程调度分析
- Linux 2.6 进程调度特点

3.1 Linux 进程调度策略

- 什么是调度策略**（scheduling policy）？它是一组规则，它们决定什么时候以怎样的方式选择一个新进程运行
- 分时技术回顾**
 - 允许多个进程“并发”运行
 - CPU 的时间被划分成“片”，给每个可运行进程分配一片
 - 在单处理器上，任何时刻只能运行一个进程，当一个并发执行的进程时间片用完时（到期）还没有终止，就可以进行进程调度
- 分时 **依赖于时钟中断，对进程透明**

3.1 Linux 进程调度策略

- 如何动态管理和维护就绪进程？
- 进程调度相关数据结构设计？
- 如何设定进程优先级？
- 进程类型：实时进程 vs. 交互式进程 vs. 普通进程
- 进程动态/静态优先级的定义
- 进程动态优先级调整策略及调整时机
- 如何设置/调整进程 CPU 时间片？
- 初始值设置原则
- 时间片修改/调整位置、时机及策略（进程创建/运行/退出）
- 进程调度/切换依据？
- 如何选择候选进程
- 与静态/动态优先级、进程类型、时间片等的关系
- 多处理器系统 SMP？
- CPU 选择策略
- 负载均衡与迁移机制
- 进程处理调度流程的设计？

调度策略设计的技术难点

3.1 Linux 进程调度策略

- 调度方法
 - 基于 **分时技术** 的 **抢占式** 调度
 - CPU 时间被划分成“**时间片**”，为每个可运行进程分配一个时间片
 - 分时依赖于 **时钟中断**，对进程透明
- 调度策略
 - 进程分类
 - 普通进程
 - 实时进程
 - 优先级
 - 静态优先级
 - 动态优先级

3.1 Linux 进程调度策略

- Linux 进程的优先级：**计算得到、动态变化**
 - 根据特定的 **算法** 计算出进程的优先级，是动态变化的，用一个 **值** 表示。
 - 调度程序会根据进程的行为周期性调整进程优先级
 - 较长时间未分配到 CPU 的进程，通常 ↑
 - 已经在 CPU 上运行了较长时间的进程，通常 ↓

3.1 Linux 进程调度策略

- Linux 既支持普通的分时进程，也支持实时进程，实时进程优先于普通进程。
- 为实时进程赋予远大于普通进程的固定的优先级。普通进程则以保证所有进程能平均占用处理器。
- 普通进程（非实时进程）**
 - 动态优先级调度：由调度程序计算出来，周期性地修改进程的优先级（避免饥饿）
- 实时进程**
 - 优先级是静态优先级，即由用户预先指定，以后不会改变
- 调度器是以进程的优先级为调度依据的，它总是选择目前就绪队列中优先级最高的进程作为候选进程 next。

3.1 Linux 进程调度策略

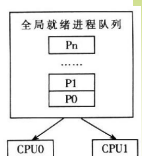
- 静态优先级**：由用户确定，只能由用户修改。系统调用 `sys_setpriority()` 改变静态优先级，
 - 实时进程** 采用静态优先级，即由用户预先指定，以后不会改变；
- 动态优先级**：只要进程占用 CPU，它的优先级就随着时间的流失而不断减小。
 - 非实时进程** 的优先级采用动态优先级，它由调度程序计算出来。

3.1 Linux 进程调度策略

- 实时进程**
 - SCHED_FIFO：先进先出。当前实时进程将一直占用 CPU 直至自动退出，除非有更紧迫的、优先级更高的实时进程需要运行时，它才会被抢占 CPU；每个可执行进程按各自在运行队列中的顺序执行，并且顺序不能变化。
 - SCHED_RR：时间片轮转。它与其它实时进程以 RR 方式去共同使用 CPU。
- 非实时进程**
 - 普通进程，SCHED_OTHER，动态优先级。
 - 时间片：用于标识一个进程在被抢占前可以运行的时间
- 用户可以通过系统调用 `sched_setscheduler()` 函数改变自己的调度策略。
- 一旦进程变为实时进程，它的子孙进程也是实时进程。

3.1 Linux 进程调度策略

- 全局就绪进程队列：由所有 **就绪态进程** 构成的全局链表的表头
 - 当前正在运行的进程也在其中（但 `idle_task` 除外）
 - 调度器从中选取最适合调度的进程投入运行
- 链表由一个读/写自旋锁保护
 - 支持多处理器并行访问
 - 但对写操作必须互斥访问



3.1 Linux 进程调度策略

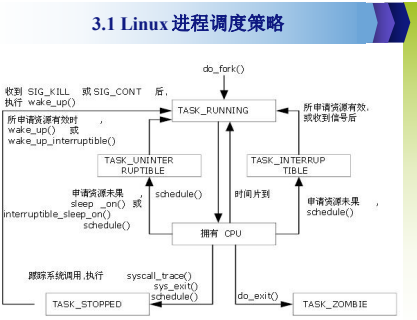
进程描述符中调度相关成员	
变量名	说明
state	进程当前状态
need_resched	布尔值，表示该进程是否需要申请调度
policy	进程调度类型
rt_priority	进程实时优先级
nice	用户控制的进程优先级
priority	静态优先级，实时进程忽略该成员
counter	当前时间片内该进程的剩余运行时间
cpus_allowed	以位向量形式表示可执行该进程的 CPU
cpus_runnable	以位向量形式表示当前运行该进程的 CPU
processor	本进程当前（或最近）所在的 CPU
thread	用于保存进程执行环境（各寄存器的值及 IO 操作许可映射表）

- 3.1 Linux 进程调度策略

- ◆ 基本思想
 - 根据 **进程价值** 及 **CPU 时间需求** 对进程分级
 - 高优先级进程先运行，低优先级后运行
 - 相同优先级进程按 **轮转** 或 **FIFO** 方式进行调度
 - ◆ Linux 实现
 - 基于动态优先级的调度算法
 - 设置基本优先级，并根据需要动态调整优先级
 - 较长时间未分配到 CPU 的进程，**提高**优先级
 - 已经在 CPU 上运行较长时间的进程，**降低**优先级
 - 两组独立的优先级范围
 - **nice**（-20~19）：确定分配给进程的时间片大小
 - **rt_priority**（0~99）：实时优先级，实时进程的优先级高于普通进程

- 3.1 Linux 进程调度策略

- ◆ 域成员名
 - state
 - ◆ 状态分类
 - **运行态/就绪态**
 - **TASK_RUNNING**：正在运行或处于就绪只等待 CPU 调度
 - **被挂起状态**
 - **TASK_INTERRUPTIBLE**：可被信号或中断唤醒进入就绪队列
 - **TASK_UNINTERRUPTIBLE**：等待资源，不可被其他进程中断
 - **TASK_STOPPED**：被调试暂停，或收到 SIGSTOP 等信号
 - **不可运行态**
 - **TASK_ZOMBIE**：退出而暂未被父进程收回资源的“僵尸”进程
 - ◆ 说明
 - 调度器主要处理 **可运行** 和 **被挂起** 两种状态下的进程



- 3.1 Linux 进程调度策略

- ◆ 静态优先级： **priority**
 - 表示分配给进程的时间片
 - 指明在被迫和其他进程竞争 CPU 之前，进程允许的最大时间片
 - 只能由用户进行修改，不随时间而改变，一般通过 **nice** 设定
 - ◆ 动态优先级： **counter**
 - 进程拥有 CPU 时随时间不断减小
 - 指明在这个时间片中所剩余的时间量
 - 当小于 0 时，标记进程重新调度
 - ◆ 实时优先级： **rt_priority**
 - 确定实时进程的调度顺序，较高数值进程优先于较低数值进程
 - 非实时进程的优先级为 0，因此实时进程总优先于非实时进程

- 3.1 Linux 进程调度策略

- ◆ 普通进程
 - 基本思想： **动态优先级调度**
 - 通过更新 counter 值，周期性修改进程优先级（避免饥饿）
 - 基本过程
 - counter 变为 0 时，用 **priority** 对 counter 重新赋值。
 - 所有可运行状态进程的时间片都用完后才对 counter 重新赋值（*）。
 - 进程运行过程中，counter 值减小为其它进程提供运行机会。
 - 该机制相当于优先级在动态变化，所以称之为动态优先调度。

- 3.1 Linux 进程调度策略

- ◆ 实时进程
 - 基本思想： **静态优先级策略**
 - counter 只用来表示该进程的剩余时间片
 - 不作为衡量其是否值得运行的标准（与普通进程的区别）

- 3.1 Linux 进程调度策略

- ◆ 动态调整 优先级及时间片长度
 - 提高交互式程序的优先级，使其运行得更频繁（*）
 - 调度程序为交互式程序提供较长的默认时间片
 - 进程不一定要一次性使用完所有的时间片，可以分批使用，从而确保尽可能长时间地处于可运行状态
 - 没有时间片的进程不会再投入运行，除非等到其他所有进程都耗尽其时间片（*）

- 3.1 Linux 进程调度策略

- ◆ 域成员名
 - policy
 - ◆ 策略分类
 - 实时进程
 - **SCHED_FIFO**：先进先出调度，除非有更高优先级进程申请运行，否则该进程保持运行至退出才让出 CPU
 - **SCHED_RR**：轮转式调度，该进程被调度下来后将被放置于运行队列的末尾，以保证其他实施进程有机会运行
 - 普通进程
 - **SCHED_OTHER**：常规分时调度策略
 - 其他
 - **SCHED_YIELD**：置位时表示主动放弃 CPU

3.1 Linux 进程调度策略

- **policy**: 调度策略,区分实时进程和普通进程,可以通过 sched_setscheduler() 来改变调度策略。

```
/*
 * Scheduling policies
 */
#define SCHED_OTHER 0
#define SCHED_FIFO 1
#define SCHED_RR 2
/*
 * This is an additional bit set when we want to
 * yield the CPU for one re-schedule..
 */
#define SCHED_YIELD 0x10
```

普通的分时进程
先入先出的实时进程
循环轮转的实时进程
当一个进程自动放弃运行时设置

3.2 Linux 进程调度依据

- 基本思想
 - 选择**权值 (weight)**最大的进程
- 权值计算: **goodness()** 函数
 - 普通进程权值
 - 与 policy, priority, counter, nice 等相关
 - $\text{weight} = p \rightarrow \text{counter}$
 - 实时进程权值
 - 与 policy, rt_priority 等相关
 - $\text{weight} = 1000 + \text{rt_priority}$

3.2 Linux 进程调度依据

权值的计算——goodness() 函数

- 功能
 - 确定就绪态进程的调度顺序
 - 运行态进程每次执行 schedule() 时都要调用该函数
- 返回值类型
 - **<=1000**: 只能赋给普通进程
 - 在单处理器下,普通进程 goodness 值只使用这个范围底部的一部分
 - 在 SMP 情况下,调度程序优先照顾等待同一处理器的进程
 - **>1000**: 只能赋给实时进程
 - 实时进程 goodness 值的范围从 1001 到 1099
- 说明
 - **goodness() 不会返回负值**
 - 由于 idle 进程的 counter 值为负,所以如果使用 idle 进程作为参数调用 goodness,将返回负值,但这是不会发生的

3.2 Linux 进程调度依据

权值影响因素分析

- 普通进程
 - 进程当前时间片内所剩的 **tick** 数
 - 既代表进程优先级,又反映进程的“欠运行程度”
 - 进程上次是否在当前 CPU 上运行
 - 若是,权值增加一个常量:
($\text{weight} += \text{PROC_CHANGE_PENALTY}$)
 - 切换是否需要切换内存
 - 若不需要,则权值加 1 ($\text{weight} += 1$)
 - 用户可控优先级 nice
 - nice 越小则权值越大; ($\text{weight} += 20 - p \rightarrow \text{nice}$)
- 实时进程
 - 权值大小仅由 **rt_priority** 值决定
 - $\text{weight} = 1000 + p \rightarrow \text{rt_priority}$
 - 基准值 1000 使实时进程的权值比所有非实时进程都要大
 - 只要就绪队列存在实时进程,调度器都将优先满足其运行需要

3.2 Linux 进程调度依据

goodness() 函数分析

```
static inline int goodness(struct task_struct *p, int this_cpu, struct
mm_struct *this_mm) {
    int weight;
    weight = -1;
    if (p->policy & SCHED_YIELD)
        goto out;
    if (p->policy == SCHED_OTHER) /* *普通进程*/
        weight = p->counter;
    if (!weight) /* 进程的时间片已经用完,则直接转到标号 out */
        goto out;

#ifdef CONFIG_SMP
    /* 在 SMP 情况下,如果进程将要运行的 CPU 与进程上次运行的 CPU 是一
    样的,则最有利,因此,假如进程上次运行的 CPU 与当前 CPU 一致的话,
    权值加上 PROC_CHANGE_PENALTY, 这个宏定义为 20. */
    if (p->processor == this_cpu)
        weight += PROC_CHANGE_PENALTY;
#endif
}
```

3.2 Linux 进程调度依据

goodness() 函数分析

```
/* 进程 p 与当前运行进程是同一个进程的不同线程,或者是
共享地址空间的不同进程,优先选择,权值加 1 */
if (p->mm == this_mm || !p->mm)
    weight += 1;
weight += 20 - p->nice;
goto out;
}
weight = 1000 + p->rt_priority; /* 实时进程 */
out:
return weight;
/* 返回值作为进程调度的唯一依据,谁的权值大,就调度
谁运行 */
```

3.2 Linux 进程调度依据

- 调度算法将 CPU 时间划分为 **时期 (epoch)**
 - 在一个单独时期内,每个进程分配一个指定时间片
 - 一个进程用完其时间片时,将会被抢占
 - 只要进程的时间片没有用完,就可以被多次调度运行
 - 所有进程用完其时间片,一个时期才结束
 - 此时才重新计算所有进程的时间片,并开始一个新时期
- Linux 的时间单位
 - 时钟滴答 (10ms)
 - 时间片即指时间滴答数
 - 例如,若 priority 为 20,则分配给该进程的时间片就为 20 个时钟滴答,即 $20 \times 10\text{ms} = 200\text{ms}$

3.2 Linux 进程调度依据

- 时间片的长短对系统性能非常关键,时间片大小的选择总是一种 **折衷**,既不能太长也不能太短
 - 太短
 - 频繁切换会造成系统开销过大
 - 假如切换时间为 1ms,时间片设置为 1ms,则没空执行进程
 - 太长
 - 几乎每个进程都一次运行完,并发的概念基本消失
 - 导致系统对交互的响应表现欠佳
 - 普通进程需要等待很长时间才能运行
- Linux 采取单凭经验的方法
 - 选择尽可能长的时间片,同时能保持良好响应时间

3.2 Linux 进程调度依据

- 为非 SCHED_FIFO 调度策略的每个进程提供一个运行的时间配额 **counter**
 - 剩余的时间片
 - 每个进程的 counter 初值与 nice 值有关
 - 每次时钟中断 (tick) 发生,时间片都会减 1,直到为 0 (则请求调度)
 - 新时期 (epoch) 开始时,将重新计算
 - 创建新进程时,子进程继承父进程的一半剩余时间片
- ```
p->counter = (current->counter + 1) >> 1;
current->counter >>= 1;
```

### 3.2 Linux 进程调度依据

- 除init进程外，所有其他进程都通过 do\_fork() 创建
- do\_fork() 与新进程调度相关的属性设置
  - state
    - TASK\_UNINTERRUPTIBLE
  - cpus\_runnable
    - 全1，未在任何cpu上运行
  - processor
    - 与父进程的processor相同
    - 优先在创建子进程的CPU上运行
  - counter
    - 父进程counter值加1的一半，父进程counter也减半
    - 子进程结束运行时，剩余时间片也将归还给父进程，以免父进程因创建子进程而遭受时间片的损失

### 3.2 Linux 进程调度依据

#### counter值的更新

- 每次产生时钟中断时，递减当前进程的 counter 值
- 更新函数：update\_process\_times()
- 调度策略为 SCHED\_RR 的进程
  - 一旦counter降到0，从runqueue中当前的位置移到队列的末尾，同时恢复其最初的时间配额
- 调度策略为 SCHED\_OTHER 的进程
  - 当所有就绪进程的 counter 值降为0时，重新计算并设置每个进程的时间配额。

### 3.2 Linux 进程调度依据

```
void update_process_times(int user_tick)
{
 struct task_struct *p = current;
 int cpu = smp_processor_id(), system = user_tick ^ 1;

 update_one_process(p, user_tick, system, cpu);
 if (p->pid) {
 if (--p->counter <= 0) {
 p->counter = 0;
 /*
 * SCHED_FIFO is priority preemption, so this is
 * not the place to decide whether to reschedule a
 * SCHED_FIFO task or not - Bhavesh Davda
 */
 if (p->policy != SCHED_FIFO) {
 if (p->needs_resched == 1)
 p->needs_resched = 0;
 }
 if (p->nice > 0)
 kstat.per_cpu_nice[cpu] += user_tick;
 else
 kstat.per_cpu_user[cpu] += user_tick;
 kstat.per_cpu_system[cpu] += system;
 } else if (local_bh_count(cpu) || local_irq_count(cpu) > 1)
 kstat.per_cpu_system[cpu] += system;
 }
 end update_process_times?
}
```

### 3.2 Linux 进程调度依据

#### 非实时进程的 counter 值更新代码

```
repeat_schedule:
/*
 * Default process to select..
 */
next = idle_task(this_cpu);
c = -1000;
list_for_each(tap, runqueue_head) {
 p = list_entry(tap, struct task_struct, run_list);
 if (can_schedule(p, this_cpu)) {
 int weight = goodness(p, this_cpu, prev->active_nice);
 if (weight > c)
 c = weight, next = p;
 }
}
/* Do we need to re-calculate counters? */
if (unlikely(!c)) {
 struct task_struct *p;

 spin_unlock_irq(&runqueue_lock);
 read_lock(&tasklist_lock);
 for_each_thread(p) {
 p->counter = (p->counter >> 1) + NICE_TO_TICKS(p->nice);
 read_unlock(&tasklist_lock);
 spin_lock_irq(&runqueue_lock);
 goto repeat_schedule;
 }
}
```

### 3.2 Linux 进程调度依据

- 当所有就绪进程的动态优先级都为 0 时，系统将重新计算进程的动态优先级，这既包括了就绪态进程，也包括了等待态进程。
  - 计算方法为：将每个进程的动态优先级的值右移一位，再加上该进程的静态优先级的值，就得到了该进程的动态优先级的新值。  
nice 缺省为 0（在 -20 到 19 之间选择）
- $p->counter = (p->counter >> 1) + NICE\_TO\_TICKS(p->nice);$
- #define NICE\_TO\_TICKS(nice) (TICK\_SCALE(20-(nice))+1)
- #define TICK\_SCALE(x) ((x) >> 2)
- 通常，基本时间片的值为 6，由于时钟中断大约 10ms 左右，因此基本时间片的长度大约 60ms
- 处于等待态的进程的动态优先级通常会逐渐增加，也可能不变，但不可能减小。

### 3.2 Linux 进程调度依据

- 对于就绪态进程：由公式可看出，由于其动态优先级都为 0，计算结果为 nice 值。
- 对于等待态进程：此时它们的动态优先级都不为 0，所以，计算结果等待态进程的动态优先级会大于静态优先级，但决不会比静态优先级两倍更大。因此，等待态进程会有较高的优先级。
- 因此，I/O 较多的进程会有较高的优先级（linux 偏向 I/O 密集型进程）；计算较多的进程因其占用 CPU 多使优先级较低。

### 3.3 Linux 进程调度时机

- 当前进程所需资源无法满足而必须立即阻塞时，直接调用 schedule()。
- 步骤
  - 将 current 进程插入到合适的等待队列中。
  - 将 current 进程当前状态改变为 TASK\_INTERRUPTIBLE 或 TASK\_UNINTERRUPTIBLE
  - 调用 schedule()
  - 检查所需资源是否可用
    - 若不可用，转到第 2 步
  - 一旦所需资源可用，将 current 从等待队列中删除。
- 说明
  - 长时间执行重复任务的很多设备驱动程序也直接调用调度程序
  - 在每次反复循环中，驱动程序都检查 need\_resched 的值，如果必要，调用 schedule() 主动放弃 CPU。

### 3.3 Linux 进程调度时机

- 将当前进程的 need\_resched 设为 1 来请求调度
- 执行时机
  - 当前进程用完 CPU 时间片时
    - 通过 update\_process\_times() 函数进行
  - 一个进程被唤醒，且优先级高于当前进程时
    - 在 wake\_up\_process() 函数中调用 reschedule\_idle()
    - if (goodness(current, p) > goodness(current, current))
    - current->need\_resched = 1;
  - 调用 sched\_setscheduler() 或 sched\_yield() 系统调用时
  - 每次进入用户态进程前
    - 都会检查 need\_resched，决定是否调用函数 schedule()
  - 系统调用执行结束，控制由核心态返回用户态前
    - 在 ret\_from\_sys\_call 入口检查当前进程的 need\_resched

### 3.3 Linux 进程调度时机

- ret\_from\_sys\_call 入口代码

```
ENTRY(ret_from_sys_call)
cli
cmpl $0, need_resched(%ebx) #ebx中存放current指针
jne reschedule
.....
reschedule:
call SYMBOL_NAME(schedule)
jmp ret_from_sys_call # 反复查询 need_resched
```

### 3.3 Linux 进程调度时机

#### 调度器相关函数和宏

- `schedule()`
  - 进程调度主函数
- `switch_to()`
  - 由 `schedule()` 调用, 进行上下文切换的宏
- `reschedule_idle()`
  - 在 SMP 系统中
    - 若被切换进程仍可运行, 则调用 `reschedule_idle()` 重新调度
    - 选择一个空闲或执行低优先级进程的 CPU 来运行被切换进程
- `goodness()`
  - 优先级计算函数, 选择一个最合适的进程投入运行

### 3.3 Linux 进程调度时机

#### `schedule()` 函数主要工作

- 从待切换进程的 `processor` 域读取 CPU 标识, 存入 `this_cpu`
- 初始化 `sched_data` 变量, 指向当前处理器 `schedule_data` 结构
- 调用 `goodness()` 函数选取合适进程
- 若需要, 重新计算动态优先级
- 将 `sched_data` → `last_schedule` 值修改为当前时间
- 调用 `switch_to()`, 切换进程上下文
- 调用 `schedule_tail()`
  - 若 `prev` 所指向进程仍可运行且不是空闲进程
    - `schedule_tail()` 调用 `reschedule_idle()` 为其选择合适 CPU
- 阅读 [schedule](#)

### 3.3 Linux 进程调度时机

#### `schedule()` 主要工作

- (1) **选择进程**: 当 CPU 空闲时, 调度子程序扫描运行队列所有活动进程, 从运行队列中选择一个合适进程运行:
  - 如果执行队列中有实时进程, 则选择一个优先级最高 (`goodness()` 计算的值最大) 的进程 (可保证实时进程总比普通进程优先执行)。
  - 如果运行队列中的进程有相同优先级, 则耗时多 (`counter` 值较大) 的进程比耗时少的进程优先调度。
  - 如果优先级相同且 `counter` 值也相等, 则位于队列前面的进程被优先调度。

### 3.3 Linux 进程调度时机

#### `schedule()` 主要工作

#### (2) 切换进程

- 当 CPU 从运行队列中选中一个合适的进程后, 就按照 `task_struct` 结构中的信息进行进程上下文切换;
- 首先把当前进程放到适当的等待队列里
- 把当前进程的状态设为 `TASK_INTERRUPTIBLE` 或者 `TASK_UNINTERRUPTIBLE`
- 调用 `schedule()`, 让新的进程运行

### 3.4 Linux 进程调度实现

- **实现**: 具体是通过 `sched.c` 文件来完成的。它的大致流程分成:
  - (1) 进程的选择和进程的切换。
    - 其中进程的选择就是遍历运行队列, 找到一个优先级最高的进程。部分代码如下:

```
repeat_schedule:
next = init_tasks[this_cpu];
c = -1000;
list_for_each(tmp, &runqueue_head) {
 p = list_entry(tmp, struct task_struct, run_list);
 if (p->cpu_running & p->cpu_allowed & (1 << this_cpu)) {
 int weight = goodness(p, this_cpu, prev->active_mm);
 if (weight > c)
 c = weight, next = p;
 }
}
```

### 3.4 Linux 进程调度实现

- 其中的 `goodness()` 是用来计算当前进程的优先级的, 用来衡量一个处于可运行状态的进程值得运行的程度。它给每个处于可运行状态的进程赋予一个 **权值 (weight)**。
- 调度程序以这个权值作为选择进程的唯一依据。选择一个权值 (`weight`) 最大的进程
- 它的第一个参数就是待估进程的描述符, 返回值比较真实的反映了待估进程 “值得运行的程度”。
- `c=0` 则表示待估进程的时间片已经用完。

### 3.4 Linux 进程调度实现

#### 进程调度中的 `prev` 指针

- 功能
  - 保存当前进程, 即可能被切换出去的进程
  - 对实时进程 (`SCHED_RR`)
    - 仅当进程时间片结束后才切换到别的进程, 此时将根据 `nice` 值重置 `counter`, 并将该进程置于就绪队列的末尾
  - 对处于 `TASK_INTERRUPTIBLE`、有信号需处理的进程
    - 不立即执行该进程, 而是将该进程置为 **就绪态**, 参与 `goodness` 选择
  - `prev` 不处于就绪态, 也不处于有信号等待处理的挂起态
    - 将其从就绪队列中删除
    - 此后, 除非有唤醒操作将其重新放回就绪队列, 否则不参与调度
  - 被动方式启动调度器工作时, 当前进程的 `need_resched` 属性置位
    - `schedule()` 将清除该位, 表示该进程已在调度器中得到处理

### 3.4 Linux 进程调度实现

#### 进程调度中的 `next` 指针

- 功能
  - 保存待切换入的新进程
  - 新进程刚好是需切换出去的老进程, 不需进行进程切换
  - 新进程运行环境实际上主要指 **内存**
    - `task_struct` 与调度相关的内存属性: **mm** 和 **active\_mm**
      - 前者指向进程所拥有的内存区域, 后者指向进程实际使用的内存
    - 对于大多数进程, **mm** 和 **active\_mm** 相同
      - 但内核线程没有自主内存, 其 `mm` 指针永远为 `NULL`
    - 调度器判断 `next-mm` 是否为空, 即可判断该进程是否为**核心线程**
      - 若是, 则继续使用 `prev` 的 `active_mm`, 并将 `cpu_tlbstate[cpu].state` 置为 `TLBSTATE_LAZY`, 表示内存管理部件不要刷新 TLB
      - 否则, 调用 `switch_mm()` 函数进行内存的切换
    - 设置好 `next` 内存环境后, 可调用 `mmdrop()` 释放 `prev` 的内存结构
      - 所有不在运行中的进程, 其 `active_mm` 属性都应该为空

### 3.4 Linux 进程调度实现

```
asmlinkage void schedule(void)
{
 struct schedule_data * sched_data;
 struct task_struct *prev, *next, *p;
 int this_cpu, c;

 spin_lock_prefetch(&runqueue_lock);
 BUG_ON(!current->active_mm);
 need_resched back:
 prev = current;
 this_cpu = smp_processor_id();
 if (unlikely(in_interrupt())) {
 printk("Scheduling in interrupt\n");
 BUG();
 }
 release_kernel_lock(prev, this_cpu);
}
```

### 3.4 Linux 进程调度实现

```
/* 'sched_data' is protected by the fact that we can run
 * only one process per CPU.
 */
sched_data = & aligned_data[this_cpu].schedule_data;
spin_lock_irq(&runqueue_lock);

/* move an exhausted PR process to be last */
if (!entroid(prev->policy == SCHED_PR))
 if ((prev->counter) <= 0) {
 prev->counter = MICE_TO_TICKS(prev->nice);
 move_last_runqueue(prev);
 }

switch (prev->state) {
 case TASK_INTERRUPTIBLE:
 if (signal_pending(prev)) {
 prev->state = TASK_RUNNING;
 break;
 }
 default:
 del_from_runqueue(prev);
 case TASK_RUNNING:
 prev->need_resched = 0;
}
```

### 3.4 Linux 进程调度实现

switch\_to() 主要工作

- 将esi, edi, ebp压入堆栈
- 堆栈指针 esp 保存为 prev → thread.esp
- 将 esp 恢复为 next → thread.esp
- 将标号 1: 的地址保存为 prev → thread.eip
- 将 next → eip 压入堆栈
- 无条件跳转到 \_\_switch\_to() 函数
  - 切换LDT和FS, GS等寄存器
  - 函数返回时已经切换到 next上下文
- switch\_to() 中 jmp 指令以后代码 (标号 1: 的代码)
  - 工作过程与第一步相反, 即从堆栈弹出 ebp, edi 和 esi
  - 该代码是下次该进程运行时最先执行的代码

### 3.4 Linux 进程调度实现

Linux 2.4 进程调度机制小结

- 进程就绪队列管理
  - 全局共享队列: **runqueue**
- 进程类型
  - 实时进程: **SCHED\_FIFO/SCHED\_RR**
  - 普通进程: **SCHED\_OTHER**
  - 其他: **SCHED\_YIELD**
- 进程优先级
  - 静态优先级: **priority**
    - 可通过 nice 调整, 决定进程的初始时间片
  - 实时优先级: **rt\_priority**
    - 只对实时进程有效
  - 动态优先级: 通过 **goodness()** 完成, 决定进程调度顺序
    - 与 policy, priority, rt\_priority, counter, nice 等相关
    - 动态优先级在 **goodness()** 中集中式完成

### 3.4 Linux 进程调度实现

Linux 2.4 进程调度机制小结

- 进程CPU时间片: **counter**
  - 初始值与 nice 有关
  - 时间片调整策略: 进程创建 / 运行 / 退出
- 进程调度 / 切换时机
  - 切换依据
    - 实时进程: **rt\_priority**
    - 普通进程: **goodness()** 计算结果
  - 切换方式
    - 主动式: 直接调用 **schedule()**
    - 被动式: **need\_resched** 设为 1
      - 使用完CPU时间片
      - 高优先级进程被唤醒 (SMP): **reschedule\_idle()**
      - 主动让出CPU: **sched\_yield()**
- 与SMP相关处理
  - **reschedule\_idle()**
  - **sched\_tail()**

### 3.4 Linux 进程调度实现

counter 小结

- 在每次时钟中断时都要递减当前进程的 counter 值
  - **update\_process\_times()**
- 对于调度策略为 SCHED\_RR 的进程, 一旦 counter 降到 0, 就要从 runqueue 中当前的位置移到队列的末尾, 同时恢复其最初的时间配额
- 对于调度策略为 SCHED\_OTHER 的进程, 当所有进程的 counter 值降为 0 时, 重新计算并设置每个进程的时间配额

### 3.4 Linux 2.4 调度算法分析

- 在 Linux 2.4 内核里, 系统中并没有根据进程的调度策略分别管理进程, 而是把将可运行进程都集中于同一队列中, 三种调度算法在实现过程中合为一种, 使用同一段程序。
- 在 Linux 2.4 内核里, 查找最佳候选就绪进程的过程是在调度器 **schedule()** 中进行的, 每一次调度都要进行一次 (在 for 循环中调用 **goodness()**), 在所有就绪进程的时间片都耗尽以后在调度器中一次性重算。这种查找过程与当前就绪进程的个数相关, 因此, 查找所耗费的时间是  $O(n)$  级的,  $n$  是当前就绪进程个数。
- 调度动作的执行时间就和当前系统负载相关, 无法给定一个上限, 这与实时性的要求相违背。

### 3.4 Linux 2.4 调度算法分析

- 每次调度时, 调度器都要线性遍历这个队列, 另外每次重新计算 counter 所花费的时间也会随着系统中进程数的增加而线性增长
  - 系统中调度算法属于  $O(n)$ , 开销线性增长
- 当大多数就绪进程的时间片都用完, 且未重新分配时间片时
  - SMP 系统中将可能有部分处理器处于空闲状态
  - 全局就绪进程队列, 对多处理器伸缩性支持不好
  - 时间片的重算循环制约多处理器的效率

### 3.4 Linux 2.4 调度算法分析

- 空闲处理器执行时间片尚未用尽、而处于等待状态的进程时, 会导致进程开始在处理器之间 “跳跃”
  - 处理器的亲和性不好
  - 实时进程或者占用内存大的进程在处理器之间跳跃会严重影响系统的性能
- 交互式进程支持不够, 不支持内核抢占

### 3.5 Linux 2.6 进程调度

优化目标

- 提供完全  $O(1)$  调度算法
  - 不管系统有多少进程, 调度算法都必须在常数时间内完成调度
- 对SMP有良好可伸缩性
  - 每个处理器应有独立的可执行进程队列和锁机制
- 提高SMP处理器的亲和性
  - 出现负载不均衡时, 应具备在处理器间迁移进程的能力



### 3.5 Linux 2.6 进程调度



55

### 3.5 Linux 2.6 进程调度

- 56

### 3.5 Linux 2.6 进程调度



### 3.5 Linux 2.6 进程调度

- ### 优先级数组数



### 3.5 Linux 2.6 进程调度

### 3.5 Linux 2.6 进程调度



### 3.5 Linux 2.6 进程调度



### 3.5 Linux 2.6 进程调度

- ### 就绪队列状态的交换

```
struct prio_array * array = rq->active;
if (!array->nr_active) {
 rq->active = rq->expired;
 rq->expired = array;
}
```

63

### 3.5 Linux 2.6 进程调度

#### • struct prio\_array

#### 优先级数组定义

typedef struct runqueue runqueue\_t;

```
struct prio_array {
 unsigned int nr_active; /* 本进程组中的进程数 */
 unsigned long bitmap[BITMAP_SIZE]; /* 加速 HASH 表边界的映射 */
 struct list_head queue[MAX_PRIO]; /* 以优先级为索引的 HASH 表 */
};
```

- queue：指定优先级进程链表的指针
  - 如 queue[i] 是 priority 为 i 的进程的指针
- bitmap：优先级位图
  - 每一位代表一个优先级
- MAX\_PRIO：优先级数量
- BITMAP\_SIZE 的值为 5
  - 通过 5 个四字节（long 型）的整数位（160 位）作为运行队列 queue[MAX\_PRIO] 的位图掩码

### 3.5 Linux 2.6 进程调度

```
static inline int sched_find_first_bit(const unsigned long *b)
{
 if (unlikely(b[0]))
 return __ffs(b[0]);
 if (unlikely(b[1]))
 return __ffs(b[1]) + 32;
 if (unlikely(b[2]))
 return __ffs(b[2]) + 64;
 if (b[3])
 return __ffs(b[3]) + 96;
 return __ffs(b[4]) + 128;
}
```

- \_\_ffs()：确定 long 类型变量最右边的 1 的所在位

65

### 3.5 Linux 2.6 进程调度

#### • 生成 160 位的位数组

#### O(1) 级调度优先级数据结构小结

- 5 个 long 类型变量，构成一张表
- 每一位对应一个优先级
  - 该优先级若有 active 状态进程，则相应位置 1
  - 否则置 0
- 从右向左扫描 1，得到优先级最高的优先级号 k
  - 进入 queue[k] 数组
- sched\_find\_first\_bit() 函数
  - 找到位图中第 1 个置位的位，得到优先级最高的优先级号 k

66

### 3.5 Linux 2.6 进程调度

#### O(1) 调度算法执行过程

- 查找过程分解为 n 步，每步耗费时间均为 O(1) 级
- prio\_array 包含一个就绪队列数组
  - 数组索引代表优先级（共 140 级）
  - 相同优先级进程放置在相应数组元素的链表 queue 中
- 调度时直接选择 active 就绪队列中具有最高优先级的链表中的 **第一项** 作为候选进程
- **优先级计算过程** 分布到各个进程的执行过程中进行
- **位映射数组** 用于加速寻找存在就绪进程的链表，其中每位对应一个优先级链表
  - 如果该优先级链表非空，则对应位为 1，否则为 0
  - 核心还构造了一个 sched\_find\_first\_bit() 函数，快速定位第一个非空的就绪进程链表

67

### 3.5 Linux 2.6 进程调度

```
struct task_struct *prev, *next;
struct list_head *queue;
struct prio_array *array;
int idx;

prev = current;
array = rq->active;
idx = sched_find_first_bit(array->bitmap);
queue = array->queue + idx;
next = list_entry(queue->next, struct task_struct, run_list);
```

68

### 3.5 Linux 2.6 进程调度

#### Linux 2.6 调度算法特点

- 每个处理器都有独立就绪进程队列
- 各处理器可 **并行运行调度程序**
- 不同处理器上的进程可以完全并行地休眠、唤醒和上下文切换
- 进程只映射到一个处理器的就绪进程队列中
  - 不会被其他处理器选中
  - 也不会不同处理器之间跳转

69

### 3.5 Linux 2.6 进程调度

#### 对比分析

- Linux 2.4 的调度算法
  - 需要遍历可运行队列，算法 O(n)
  - Epoch，基本时间片，动态优先级
- Linux 2.6.17 的调度算法（2.6.23 之前）
  - 采用双队列（Active；expire），按照优先级组队，O(1)
- Linux 2.6.26 的调度算法
  - 非实时：CFS，vruntime，红黑树
  - 实时：优先级队列
- Linux 进程可以指定该进程所采用的调度策略
- 调度算法根据进程的调度策略，采用不同调度算法

### 3.5 Linux 2.6 进程调度

- Linux 2.4 的调度算法
  - 需要遍历可运行队列，算法 O(n)
  - Epoch，基本时间片，动态优先级
- Linux 2.6.17 的调度算法（2.6.23 之前）
  - 采用双队列（Active；expire），按照优先级组队，O(1)
- Linux 2.6.26 的调度算法
  - 非实时：CFS，vruntime，红黑树
  - 实时：优先级队列
- Linux 进程可以指定该进程所采用的调度策略
- 调度算法根据进程的调度策略，采用不同的调度算法

### 3.5 Linux 2.6 进程调度

- time\_slice
  - 2.6 中，time\_slice 变量代替了 2.4 中的 counter 变量来表示进程剩余运行时间片，但不再直接影响进程的动态优先级。
- first\_time\_slice
  - 0 或 1，表示是否是第一次拥有时间片（刚创建的进程）。这一变量用来判断进程结束时是否应当将自己的剩余时间片返还给父进程



### 3.5 Linux 2.6 进程调度

- time\_slice 对调度的影响
  - 在 2.4 中，进程剩余时间片是除 nice 值以外对动态优先级影响最大的因素，并且休眠次数多的进程，它的时间片会不断叠加，从而算出的优先级也更大，调度器正是用这种方式来体现对交互式进程的优先策略。
  - 但实际上休眠次数多并不表示该进程就是交互式的，只能说明它是 I/O 密集型的，因此，这种方法精度很低，有时因为误将频繁访问磁盘的数据库应用当作交互式进程，反而造成真正的用户终端响应迟缓。

### 3.5 Linux 2.6 进程调度

- time\_slice 对调度的影响
  - 2.6 的调度器以时间片是否耗尽为标准将就绪进程分成 active、expired 两大类，分别对应不同的就绪队列，前者相对于后者拥有绝对的调度优先权——仅当 active 进程时间片都耗尽，expired 进程才有机会运行。
  - 但在 active 中挑选进程时，调度器不再将进程剩余时间片作为影响调度优先级的一个因素，

### 本章小结

- Linux 进程调度策略
  - 普通进程、实时进程
- Linux 进程调度依据
  - counter 值的变化，counter 值重新被赋值的时机
- Linux 进程调度时机
  - 主动调度、被动调度
- Linux 进程调度实现
  - goodness(), weight
- Linux 2.4 进程调度分析
- Linux 2.6 进程调度特点